MCMP117 · WEEK 2

Fundamentals of Operating Systems

Week 2 · Definitions, Goals, Components & OS Structures

What exactly is an operating system, what is it for, and how is it built?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 1 in Four Ideas

A computer follows instructions

Input → process → output, with the program stored in memory next to the data.

The OS starts first

…and manages everything else, so that other programs can run.

Idle CPUs cost money

Batch processing, multiprogramming and time-sharing were all invented to keep the processor busy.

UNIX 1969, Linux 1991

Small, composable tools; Linux is the system we will use hands-on from Week 9.

Today we replace the informal definition with a precise one — and look at how operating systems are actually built.

Today's Agenda

1. **Definitions** What exactly is an operating system? Two classic views.
2. **Goals** Convenience, efficiency, ability to evolve — and their trade-offs.
3. **Components** Hardware, OS, applications, users; interrupts, storage, dual mode.
4. **Structures** Simple, monolithic, layered, microkernel, modular, hybrid.
5. **Hands-on** Watching system calls with real C code and strace.

PART 1

What Exactly Is an Operating System?

From an informal description to a precise definition.

DEFINITIONS

Three Definitions You Will Meet

The intermediary

A program that acts as an intermediary between the user of a computer and the computer hardware.

— Silberschatz, Galvin & Gagne

The kernel

The one program running at all times on the computer. Everything else is either a system program or an application program.

— Silberschatz, Galvin & Gagne

The manager

Software that controls the execution of programs and manages the computer's resources: processor, memory, files and devices.

— Stallings (paraphrased)

Our working definition: the operating system is the software layer that manages the hardware and offers programs a simpler, safer machine to run on. Its always-resident core is called the kernel.

TWO CLASSIC VIEWS

Extended Machine vs. Resource Manager

Andrew Tanenbaum describes the OS from two directions — top-down and bottom-up.

Top-down: an extended machine

- Hides ugly hardware behind clean abstractions.
- You see files, not disk sectors and magnetic tracks.
- You see processes, not interrupt lines and registers.
- Goal: make the computer convenient to program and use.

Bottom-up: a resource manager

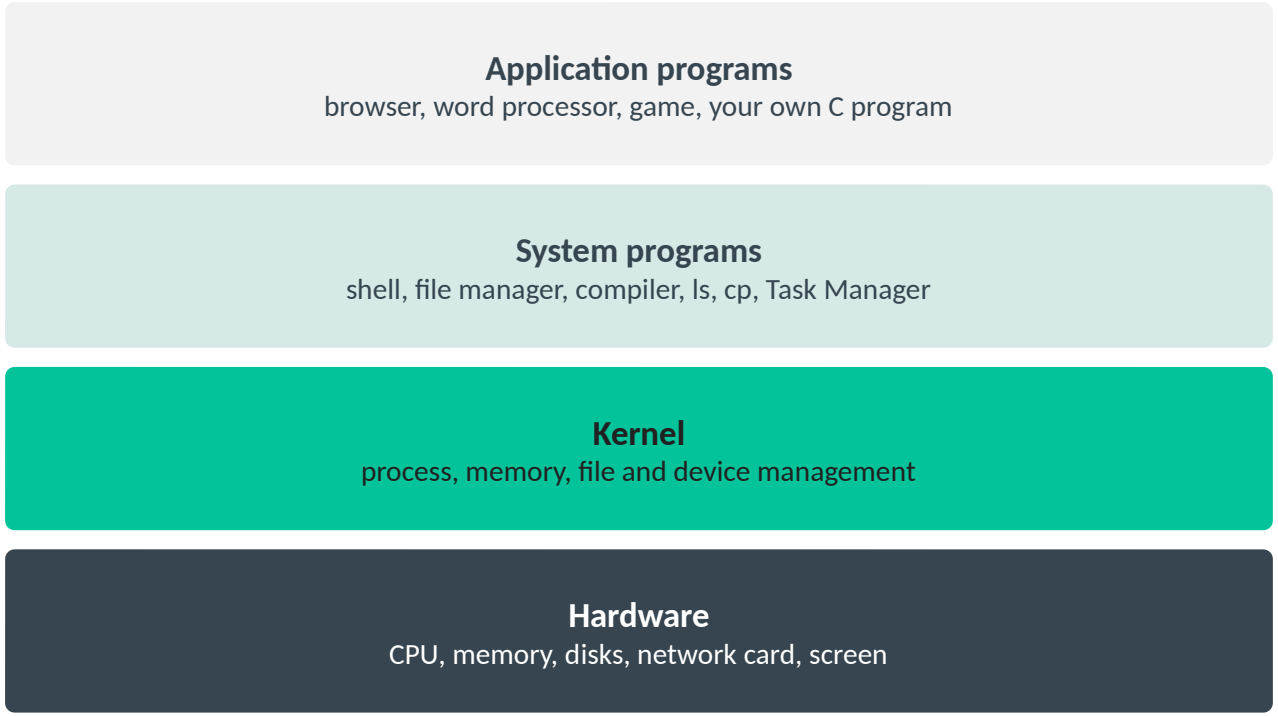
- Many programs and users compete for one machine.
- **Time multiplexing:** take turns (the CPU, a printer).
- **Space multiplexing:** share in pieces (memory, disk space).
- Goal: allocate resources fairly, efficiently and safely.

Both views describe the same software. Weeks 3–7 of this course are mostly the resource-manager view: processes, CPU, memory, files.

DEFINITIONS

Kernel, System Programs and Applications

Not everything that ships with an operating system is "the kernel".



Each layer uses only the services of the layer below it.

Who is who?

- **Kernel:** always in memory, runs in a privileged mode, talks directly to hardware.
- **System programs:** ship with the OS but run as normal programs; they make the system usable.
- **Applications:** everything you install to do your actual work.
- Linux, strictly speaking, is only the kernel; a "Linux distribution" adds system programs (mostly from GNU).

DISCUSSION

Is It Part of the Operating System?

There is no single official answer — which is exactly the point. Vote, then argue.

The web browser?

In 1998 the U.S. government sued Microsoft, partly over bundling Internet Explorer with Windows. Is the browser "part of Windows"? Lawyers argued about it for years.

The graphical desktop?

On Windows it is tightly integrated. On Linux it is a replaceable program (GNOME, KDE, …) running on top of the kernel.

The app store?

On a phone, most users think of the app store as "the system". Technically it is an application with special permissions.

A useful rule: the kernel is certainly the OS. Beyond the kernel, "the OS" is whatever the vendor ships as the standard system.

PART 2

What Is an Operating System For?

Three design goals — and why they pull in different directions.

Three Goals of Every Operating System

William Stallings summarises the objectives of an OS in three words.

1. Convenience

- Makes the computer easier to use and to program.
- Files, windows, a shell, simple system calls.
- Users should not need to know the hardware.

2. Efficiency

- Uses the hardware well: CPU busy, memory full of useful work, disks not waiting.
- Low overhead: the OS itself should use few resources.

3. Ability to evolve

- New hardware (SSDs, GPUs, 5G modems) and new services must be addable…
- …without breaking existing programs.
- Drives the choice of OS structure (Part 4).

Every structure we discuss later is a different compromise between these three goals.

TRADE-OFFS

Which Goal Matters Most? It Depends on the Machine

Kind of system	Main priority	Why
Personal computer	Convenience	One user who wants things to "just work"; the CPU is idle most of the time anyway.
Server / cloud	Efficiency	Thousands of users; 1% more throughput can mean hundreds of fewer machines.
Smartphone	Efficiency (energy)	Battery life: the OS aggressively puts apps and the CPU to sleep.
Embedded / IoT	Efficiency (size, cost)	A washing machine controller may have a few kilobytes of memory; there may be no OS at all.
Real-time (car brakes, drones)	Predictability	An answer that arrives late is a wrong answer: deadlines must always be met.

Convenience vs. Efficiency: A Worked Example

The situation

- A program must read 1,000,000 numbers from a file.
- **Convenient way:** ask the OS for one number at a time → 1,000,000 system calls.
- **Efficient way:** ask for blocks of 4,096 bytes (≈ 1,000 numbers each) → about 1,000 system calls.
- If one system call costs about 0.1 µs of pure overhead (we measure this later today):

```
one at a time:
  1,000,000 × 0.1 µs = 100 ms

in blocks of 1,000:
      1,000 × 0.1 µs = 0.1 ms

=> 1000 times less overhead
```

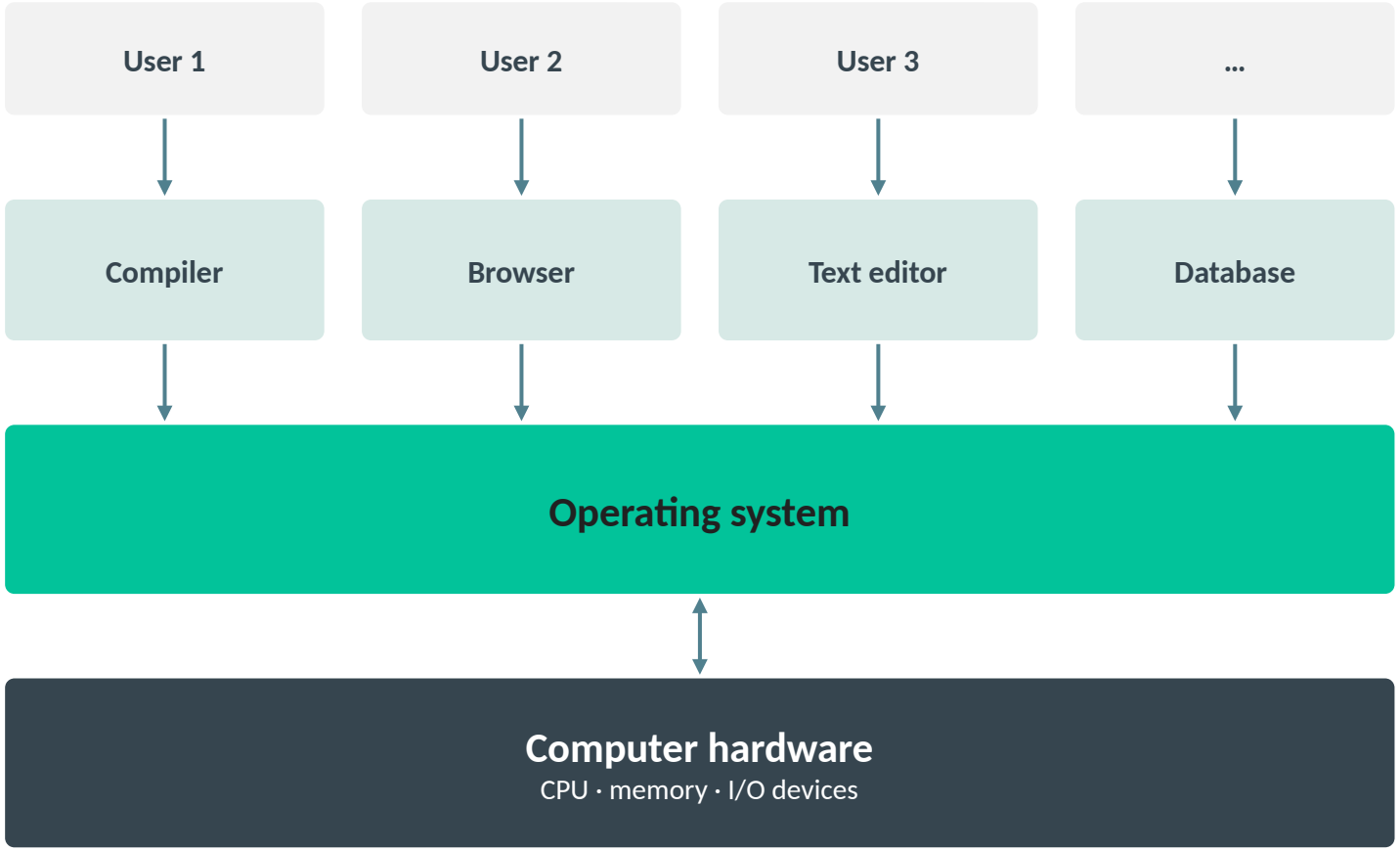
Libraries such as C's stdio give you both: you call fgetc() one character at a time, but the library reads big blocks behind the scenes.

PART 3

Components of a Computer System

Hardware, operating system, application programs and users.

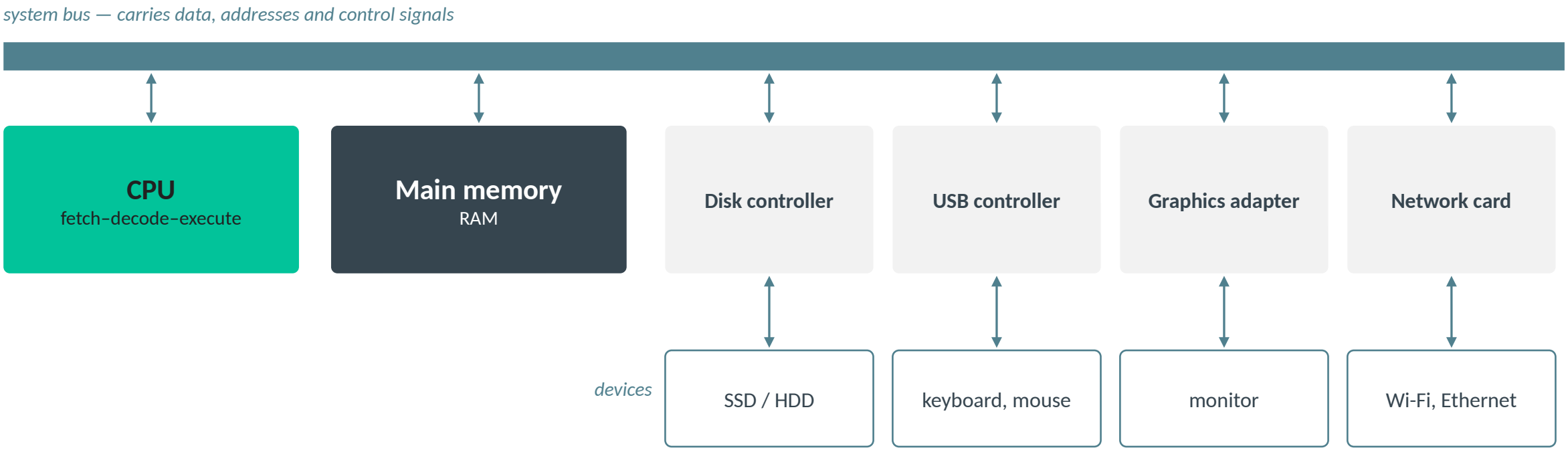
The Four Components of a Computer System



Roles

- **Hardware** provides the basic computing resources.
- **Application programs** use those resources to solve users' problems.
- **Users** — people, but also other machines and programs.
- **The OS** controls the hardware and coordinates its use among the applications and users.

Inside the Box: CPU, Memory and Devices on a Bus



Each device controller has a small local buffer and works in parallel with the CPU. The OS contains a device driver for each controller: the driver speaks the controller's language so the rest of the OS does not have to.

HARDWARE VIEW

Interrupts: How a Device Gets the CPU's Attention

The CPU does not keep asking the keyboard "anything new?" — the keyboard tells the CPU.

1. Device finishes

The disk controller has copied the data into its buffer.

→

2. Signal

It raises an interrupt line on the CPU.

→

3. Save & jump

The CPU saves what it was doing and jumps to the handler listed in the interrupt vector.

→

4. Handle

The OS's interrupt handler (part of the driver) processes the event.

→

5. Resume

The saved state is restored; the interrupted program continues as if nothing happened.

Modern operating systems are interrupt-driven: when no program is running and no device needs attention, the OS simply waits. Every key press, network packet and finished disk read arrives as an interrupt — thousands per second.

HARDWARE VIEW

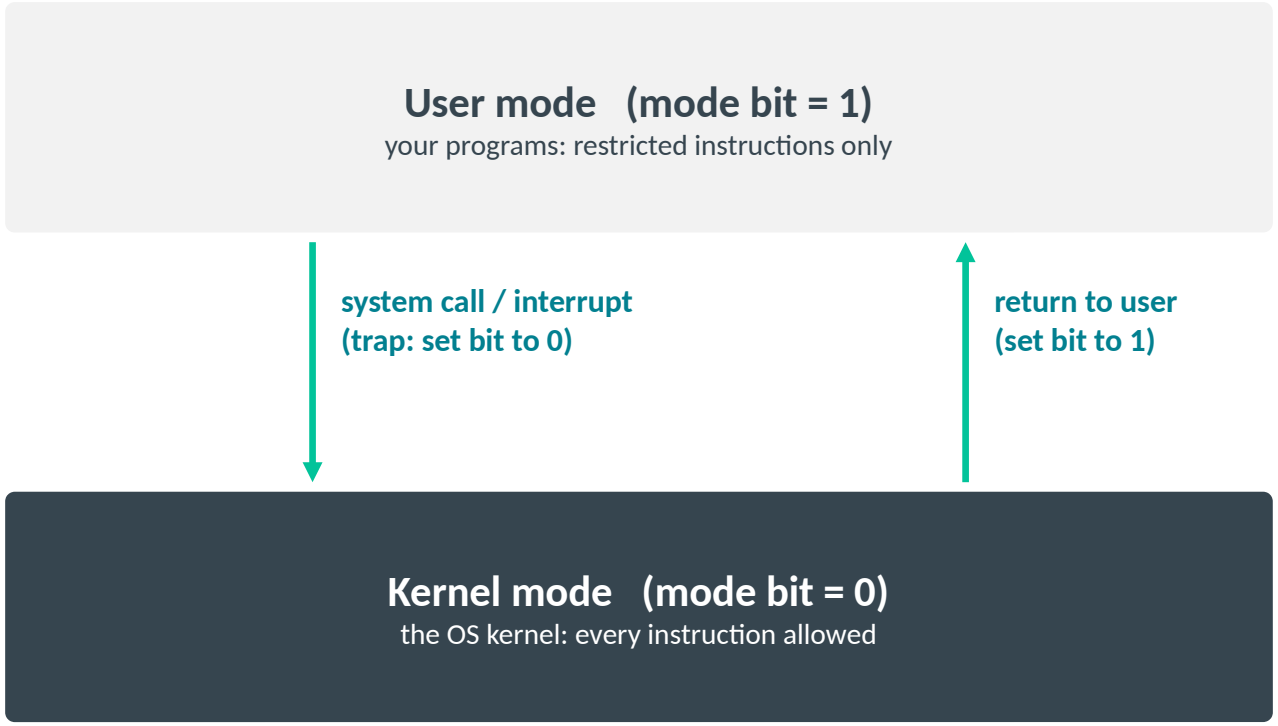
The Storage Hierarchy: Fast and Small vs. Slow and Big

Level	Typical size	Typical access time	If a register access took 1 second…
CPU registers	< 1 KB	≈ 0.3 ns	1 second
Cache (L1–L3)	a few MB	1 – 40 ns	3 seconds – 2 minutes
Main memory (RAM)	8 – 64 GB	≈ 100 ns	≈ 5 minutes
SSD	0.5 – 4 TB	≈ 50 µs	≈ 2 days
Hard disk (HDD)	1 – 20 TB	≈ 5 ms	≈ 6 months

Registers, cache and RAM are volatile (lost at power-off); SSDs and disks are non-volatile. The OS constantly moves data between levels — the theme of Weeks 6 and 7.

Dual-Mode Operation: User Mode and Kernel Mode

The hardware itself enforces who may do what, using a mode bit inside the CPU.



Why it matters

- **Privileged instructions** (talk to a device, change memory protection, halt the CPU) work only in kernel mode.
- If a user program tries one, the hardware traps to the OS, which usually kills the program.
- So a buggy or malicious app cannot take over the machine.
- On x86 CPUs the modes are called rings: Linux and Windows use ring 0 (kernel) and ring 3 (user).

HANDS-ON

Example 1: A Program That Calls the Kernel

```c
/* hello_syscall.c -- the same message printed in two ways.
 * Compile:  gcc hello_syscall.c -o hello_syscall
 * Run:      ./hello_syscall
 * Trace:    strace ./hello_syscall      (Linux only)
 */
#include <stdio.h>    /* printf: library function (user mode)   */
#include <unistd.h>   /* write:  thin wrapper around a system call */

int main(void)
{
    /* 1) Library call: printf formats the text in user mode,
     *    then asks the kernel to output it via write().      */
    printf("Hello from printf!\n");
    fflush(stdout);

    /* 2) Direct system call: file descriptor 1 = the screen
     *    (standard output). 20 = number of bytes to write.   */
    write(1, "Hello from write()!\n", 20);

    return 0;
}
```

What happens

- printf() is a library function: it formats text in user mode.
- write() is a system call: it switches to kernel mode, and the kernel sends the bytes to the screen.
- strace shows every system call a program makes:

```
$ strace ./hello_syscall
write(1, "Hello from printf!\n", 19) = 19
write(1, "Hello from write()!\n", 20) = 20
exit_group(0)                       = ?
```

HANDS-ON

Example 2: How Expensive Is a Trip into the Kernel?

```c
#define N 1000000L

long plain_function(long x) { return x + 1; }

int main(void)
{
    volatile long sink = 0;
    double t0 = seconds();
    for (long i = 0; i < N; i++)
        sink += plain_function(i);   /* user mode only */
    double t1 = seconds();
    for (long i = 0; i < N; i++)
        sink += syscall(SYS_getpid); /* user->kernel->user */
    double t2 = seconds();

    printf("ordinary function call : %6.1f ns\n",
           (t1 - t0) / N * 1e9);
    printf("system call (getpid)   : %6.1f ns\n",
           (t2 - t1) / N * 1e9);
}
/* full program: code/syscall_cost.c */
```

The experiment

- Call an ordinary function one million times.
- Then make one million tiny system calls (getpid).
- Compare the average time per call.
- Exact numbers depend on your machine; the ratio is the lesson.

```
$ ./syscall_cost
ordinary function call :   1.5 ns
system call (getpid)   : 116.1 ns
system call is about 76 times slower
```

INTERMISSION

10-Minute Break

Next: how operating systems are built — from MS-DOS to microkernels.

PART 4

Operating-System Structures

Same goals, very different buildings.

Why Structure Matters

The Linux kernel has over 30 million lines of code. How do you organise something that big?

Simple

Little structure at all (MS-DOS).

Monolithic

One big kernel program (UNIX, Linux).

Layered

Strict layers, each using only the one below (THE).

Microkernel

A tiny kernel; services run as user programs (Mach, MINIX 3, QNX).

Modular

A core kernel plus loadable modules (Linux).

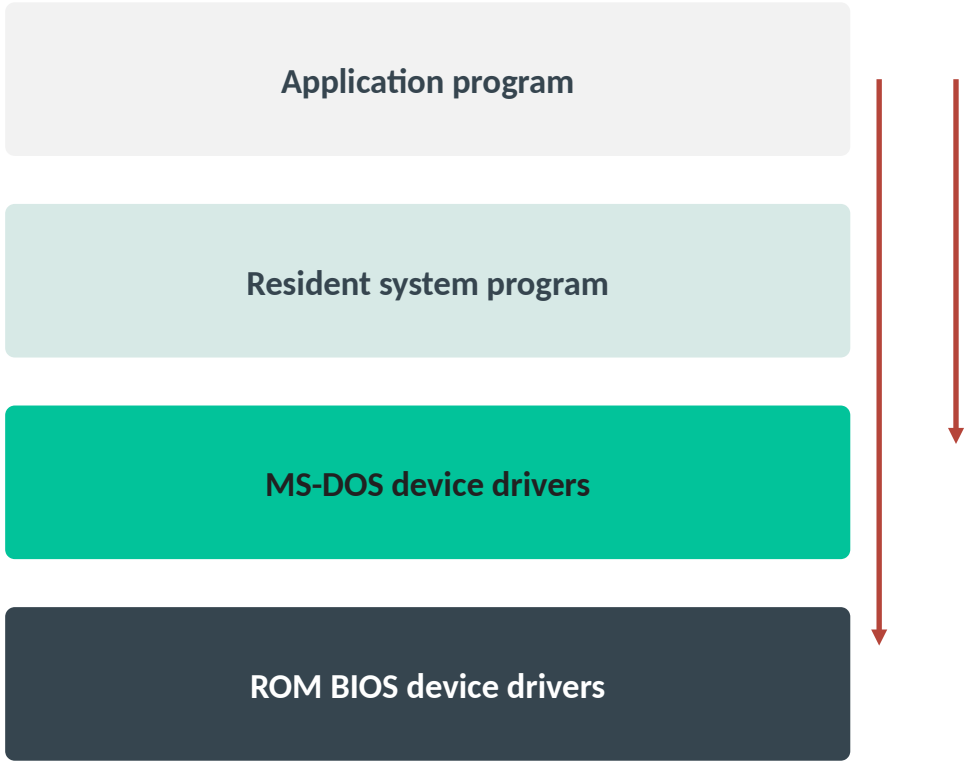
Hybrid

Real systems mix the above (Windows, macOS, Android).

STRUCTURE 1

Simple Structure: MS-DOS

Written to fit in very little memory on the 1981 IBM PC; it was never really designed in layers.



Applications could skip layers and access the hardware directly.

Advantages

- Very small and fast.
- Easy to write for limited hardware.

Disadvantages

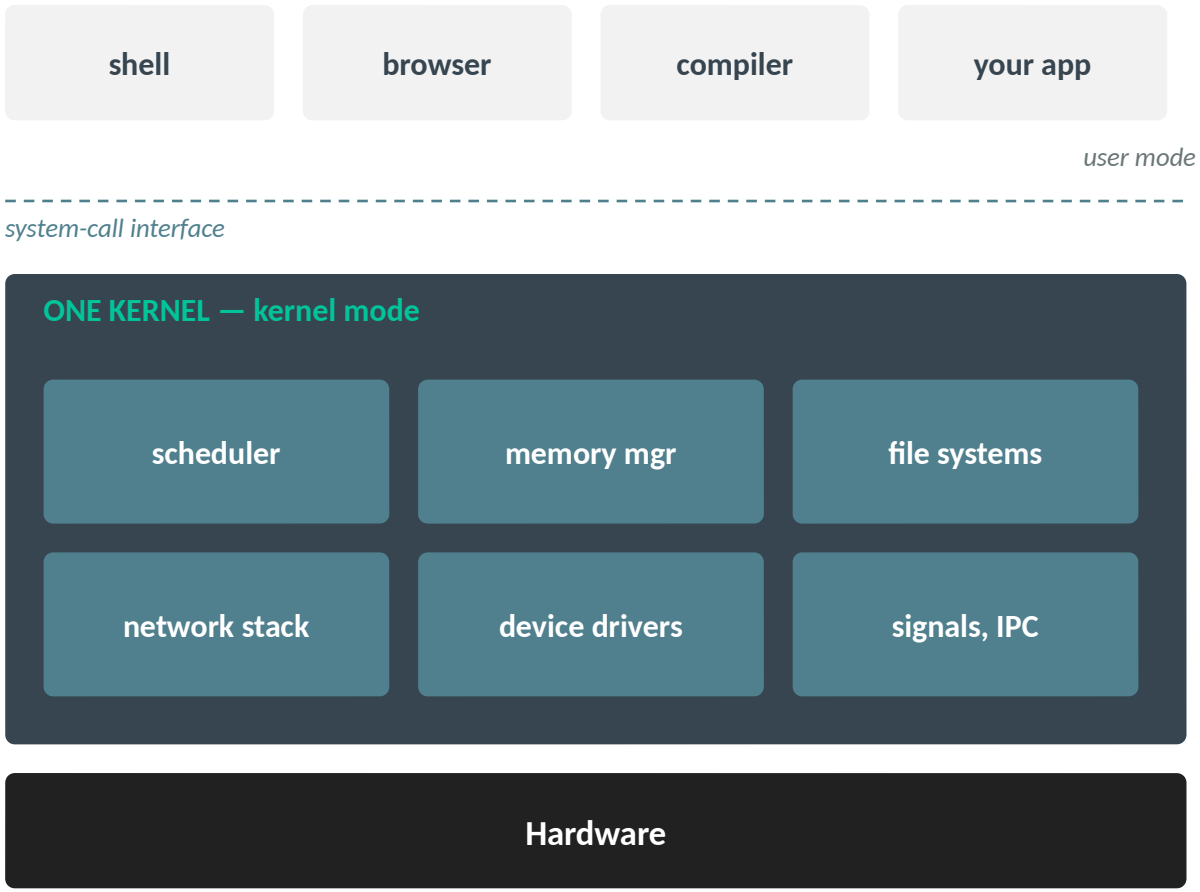
- No protection: one buggy program could crash the whole system.
- The Intel 8088 CPU had no dual mode, so the OS could not stop it.

Lesson: protection needs hardware support (dual mode) plus an OS that uses it.

STRUCTURE 2

Monolithic Kernel: UNIX and Linux

All OS services live in one large program that runs in kernel mode, in one address space.



Advantages

- Fast: services call each other directly, like ordinary function calls.
- Only two mode switches per system call.

Disadvantages

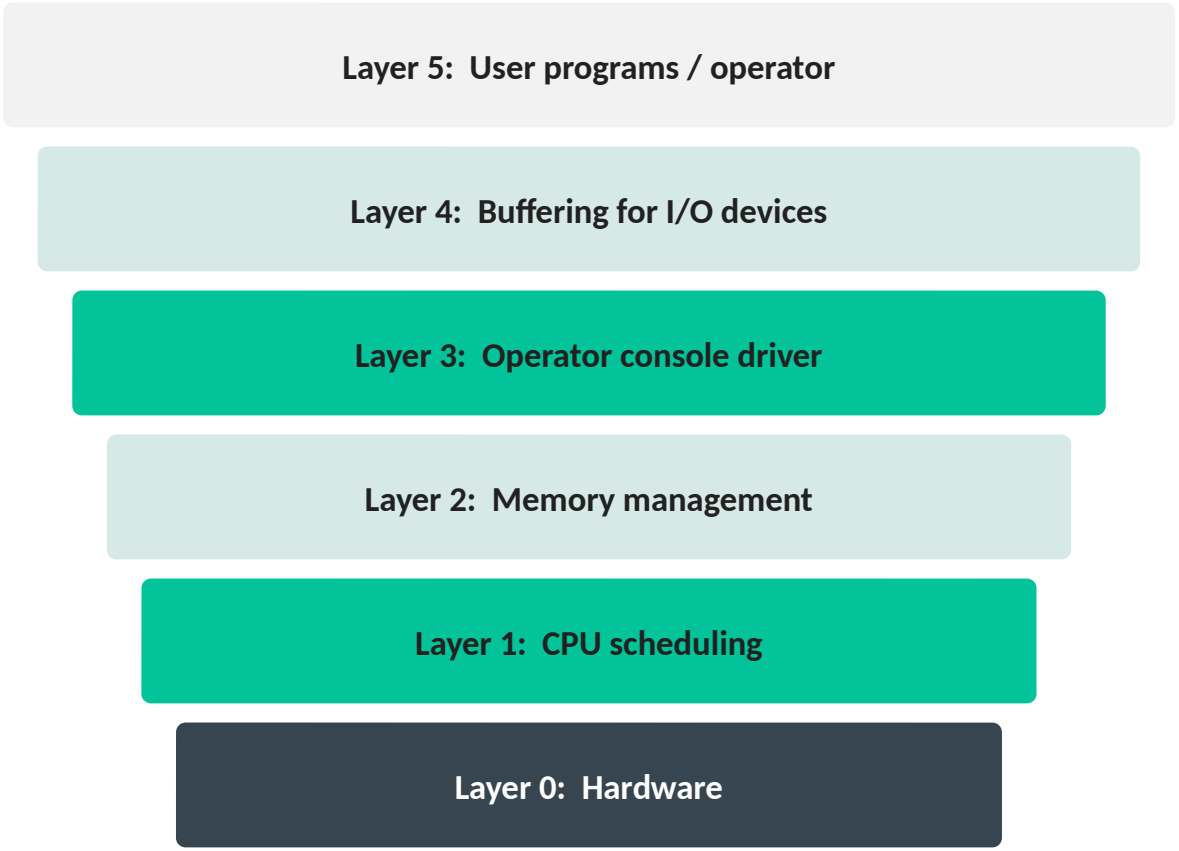
- Huge and hard to maintain.
- A bug in any driver can crash the entire kernel (a "kernel panic").

Examples: original UNIX (1970s), Linux, traditional BSD.

STRUCTURE 3

Layered Approach

The OS is split into layers; layer N may only use layers below it. Layer 0 is the hardware.



Loosely based on Dijkstra's THE system (1968).

Advantages

- Easy to build and debug one layer at a time.
- Each layer can be verified separately.

Disadvantages

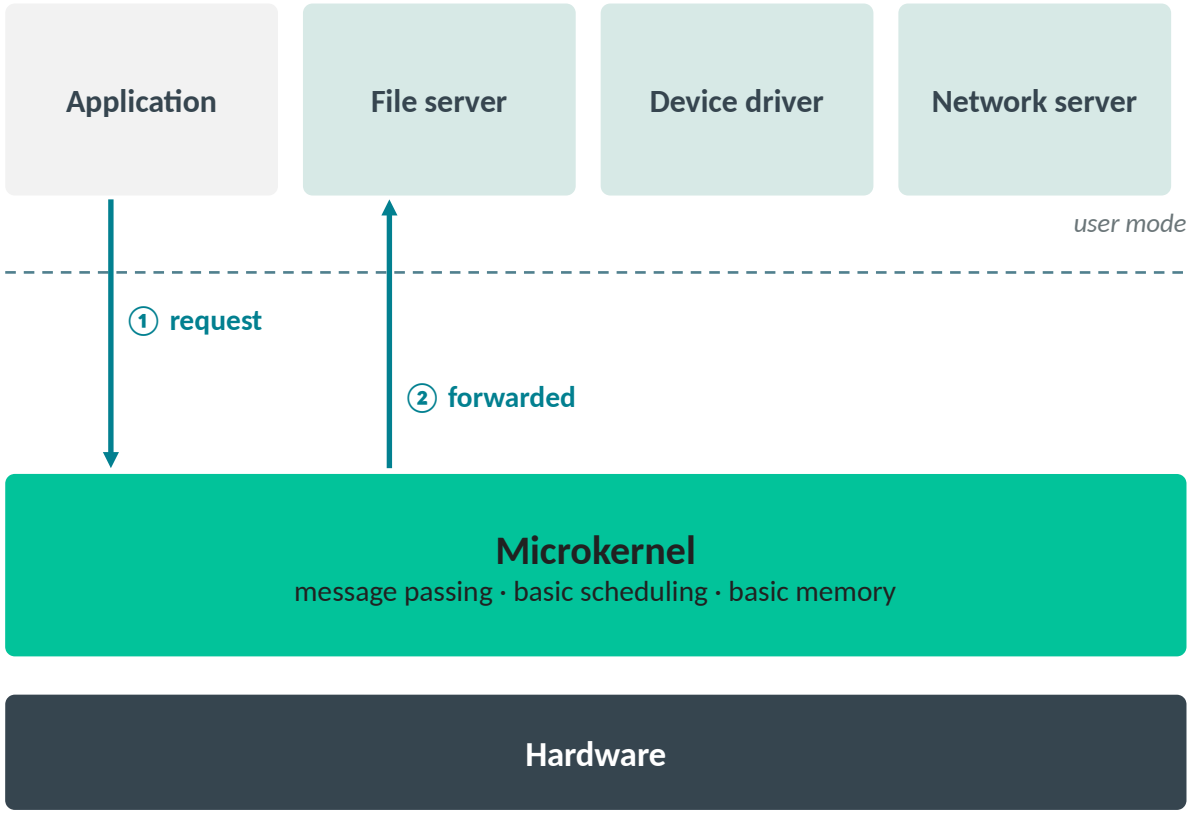
- Hard to decide the order (does memory need the scheduler, or the reverse?).
- Every request passes through many layers → slow.

Pure layering is rare today, but the idea survives inside every OS (e.g. the network stack is layered).

STRUCTURE 4

Microkernel: Keep the Kernel Tiny

Only the bare minimum stays in kernel mode; everything else runs as ordinary user-mode servers.



The app never calls the file server directly: messages travel through the kernel.

Advantages

- Reliable: a crashed driver is just a crashed program — restart it.
- Secure and easy to extend; small kernel is easier to verify.

Disadvantages

- Slower: every service request needs extra messages and mode switches.

Examples: Mach (1980s), MINIX 3, QNX (cars, medical devices), seL4 (mathematically verified).

Worked Example: The Price of a Microkernel

Counting mode switches for one file read

- **Monolithic:** app → kernel → app. 2 switches.
- **Microkernel:** app → kernel → file server → kernel → app. 4 switches, plus 2 message copies.
- Assume (illustrative numbers): one switch = 0.1 µs, one message copy = 0.15 µs, the useful work = 1 µs.

```
Monolithic:
  T = 2(0.1) + 1.0          = 1.2 µs

Microkernel:
  T = 4(0.1) + 2(0.15) + 1.0 = 1.7 µs

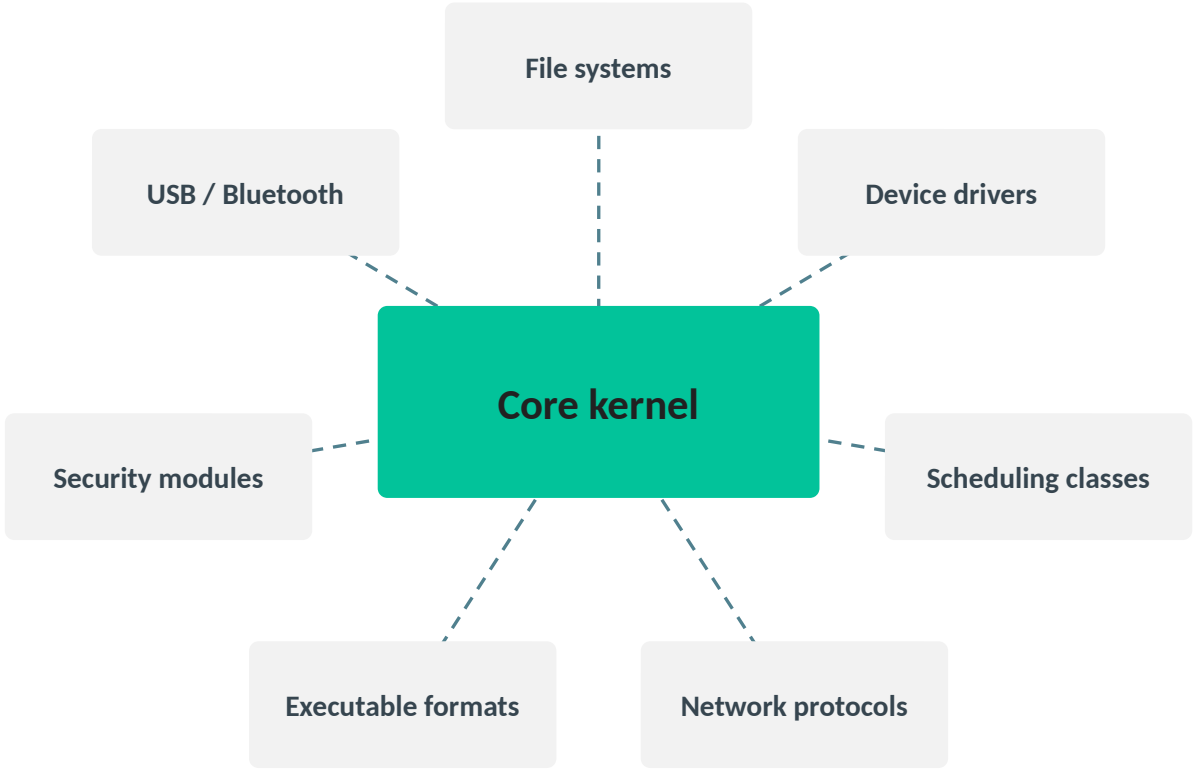
Slow-down: 1.7 / 1.2 ≈ 1.42  (42 %)
```

For short, frequent operations the overhead dominates. For long operations (reading a large file) it hardly matters. Modern microkernels such as L4 work hard to make messages cheap.

STRUCTURE 5

Modular Kernel: Loadable Kernel Modules

A core kernel plus modules that can be added or removed while the system runs.



Modules are loaded on demand and linked into the running kernel.

Advantages

- Like monolithic: modules run in kernel mode, so calls are fast.
- Like microkernel: flexible — load a driver only when the device appears.

Disadvantages

- A buggy module still runs in kernel mode and can crash the system.

Examples: Linux (lsmod, insmod, rmmod), Solaris, modern BSDs. The most common design today.

HANDS-ON

Example 3: A Loadable Kernel Module in C

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MCMP117");
MODULE_DESCRIPTION("Hello-world kernel module");

static int __init hello_init(void)      /* runs at insmod */
{
    pr_info("hello_module: loaded into the kernel\n");
    return 0;                           /* 0 = success */
}

static void __exit hello_exit(void)     /* runs at rmmod */
{
    pr_info("hello_module: removed from the kernel\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

How to use it (Linux)

- Build with the kernel's own build system (Makefile in the code folder).
- pr_info() writes to the kernel log, not to the screen.
- Only try this inside a virtual machine.

```
$ make
$ sudo insmod hello_module.ko
$ lsmod | grep hello
$ sudo dmesg | tail -1
hello_module: loaded into the kernel
$ sudo rmmod hello_module
```

STRUCTURE 6

Hybrid Systems: What Real Operating Systems Do

Pure designs are rare. Real systems borrow from several structures for performance, security and convenience.

Windows (NT family)

- Designed with microkernel ideas (separate subsystems).
- For speed, most services — including graphics — run in kernel mode.
- Loadable drivers, like modules.

macOS and iOS

- Kernel "XNU" = Mach microkernel + BSD UNIX parts, compiled into one kernel.
- Drivers as loadable kernel extensions.

Android

- A modified Linux kernel (monolithic + modules).
- On top: hardware abstraction layer, native libraries and the Android runtime for apps.

1992: Andrew Tanenbaum (MINIX, microkernel) told Linus Torvalds that "Linux is obsolete" because it was monolithic. Thirty years later, both designs are still in daily use.

SUMMARY

The Structures Side by Side

Structure	In kernel mode	Speed	Reliability	Example
Simple	everything (no protection)	high	very low	MS-DOS
Monolithic	all OS services	high	medium	UNIX, Linux
Layered	all layers	low	high	THE (1968)
Microkernel	only the tiny kernel	lower	high	MINIX 3, QNX, seL4
Modular	core + loaded modules	high	medium	Linux, Solaris
Hybrid	varies	high	medium–high	Windows, macOS, Android

The recurring trade-off: every boundary you add (layer, message, mode switch) buys reliability and costs speed.

TRY IT YOURSELF

Look Inside Your Own Operating System

```
# Linux (lab PCs, WSL, or a virtual machine)
$ uname -r          # which kernel version?
$ lsmod | head      # which modules are loaded?
$ strace -c ls      # count the system calls of ls

# Windows
> winver            # which Windows build?
> driverquery       # list installed drivers
# Task Manager > Performance: CPU time split
# into "user" and "kernel" time
```

Questions to answer

- How many kernel modules are loaded on your machine?
- Which system call does ls use most often?
- Roughly how many system calls does a simple command like ls make? (On our test machine: 76.)
- Open Task Manager and find a program that spends a lot of time in kernel mode. Why might that be?

WRAP-UP

Week 2 — Key Takeaways

- **Definition:** the OS manages the hardware and offers programs a simpler, safer machine; its always-resident core is the kernel.
- **Two views:** extended machine (abstraction, top-down) and resource manager (multiplexing, bottom-up).
- **Three goals:** convenience, efficiency, ability to evolve — weighted differently on PCs, servers, phones and embedded systems.
- **Four components:** hardware, OS, application programs, users.
- **Protection needs hardware:** dual mode (user/kernel) and interrupts; a system call costs far more than a function call.
- **Structures:** simple, monolithic, layered, microkernel, modular, hybrid — each a different speed vs. reliability compromise.

QUICK QUIZ

Check Your Understanding

1. Give one example of time multiplexing and one of space multiplexing performed by an OS.

2. Why could a bug in an MS-DOS program crash the whole computer, but not in a Linux program?

3. A system call takes 100 ns, a function call 1.5 ns. How much overhead do 10^6 system calls add, compared with 10^6 function calls?

4. Name one advantage and one disadvantage of a microkernel compared with a monolithic kernel.

NEXT WEEK

Services, System Calls & Processes

Week 3 will cover:

- The services an operating system provides
- System calls in detail: types, parameters, the system-call table
- The process concept: a program in execution
- Process states: new, ready, running, waiting, terminated

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz